Class-Based vs. Function-Based Views

Django had initially begun with FBVs, and later the CBVs were added, which aided in templatising the views so that you don't need to write code again and again.

At the core, CBVs are Python classes. Django deals with a variety of CBVs with pre-configured functionality that can be reused and often extended. There is also the term 'generic class-based views', which provide solutions to common requirements.

A function-based View takes a web request and gives a web response in the form of a web page, an XML document, a redirect 404 error, or an image. The view consists of the logic that returns the response.

This function is easy to implement but has a drawback: In a large size Django project, there are many similar functions in the views. The objects of a Django project have CRUD operations, which makes the code repeat in a loop. This is why the generic class-based views were introduced and are explained as follows.

Generic Class-Based Views

The generic class-based-views handle the common use cases of a web application, such as creating objects, listing and archiving views, handling forms, separating digital content into discrete pages, and much more.

They come in the Django core and are implemented from the django.views.generic module.

They can workably speed up the development process as they are an advanced set of Built-in views used for implementing selective view strategies, such as Create, Retrieve, Update, and Delete.

A simple Home Page View Program

The home page is the simplest view program and is used to display an HTML page. A generic CBV is used here with the TemplateView functionality.

Steps for Creating the Home Page View

1. Open Views.py file in the IDE
2. from django.views.generic.base import TemplateView
3. class HomePage(TemplateView):
4. template_name = 'marketing/home.html'
5. Commit changes